Taller 2 – Aplicación Web Full Stack

Objetivo

Desarrollar una aplicación web full stack utilizando Angular para el frontend y FastAPI para el backend, integrando persistencia de datos, autenticación de usuarios y consumo de una API REST.

Descripción

Cada equipo deberá desarrollar una aplicación web de **temática libre**. La aplicación deberá permitir gestionar información relacionada con el tema seleccionado y contar con usuarios registrados.

La solución deberá estar dividida en:

- **Frontend:** Angular
- **Backend:** FastAPI.
- **Base de datos:** PostgreSQL o MySQL.
- **Comunicación:** API REST utilizando JSON.

Requerimientos

1. Aplicación Angular

El frontend deberá desarrollarse utilizando Angular y deberá incluir:

- Al menos **4 vistas o páginas** navegables mediante Angular Router.
- Componentes reutilizables para elementos utilizados en distintas partes de la aplicación.
- Servicios para separar responsabilidades y lógica compartida.
- Consumo de la API de FastAPI utilizando las herramientas HTTP de Angular.
- Uso de **Signals** para manejar al menos un estado relevante de la aplicación.
- Uso de al menos una de las APIs `resource()`, `rxResource()` o `httpResource()`.
- Uso de **Reactive Forms** y validaciones para los formularios principales.
- Uso de al menos un **Guard** para proteger rutas.
- Uso de al menos un **Interceptor** para modificar o procesar solicitudes HTTP.

2. Angular Material

La interfaz deberá desarrollarse utilizando **Angular Material**. Se deberán utilizar componentes de Angular Material de acuerdo con las necesidades de la aplicación. Por ejemplo:

- `MatToolbar` y/o `MatSidenav` para navegación.
- `MatCard` para presentar información.
- `MatFormField` y controles Material en formularios.
- `MatButton` para acciones.
- `MatTable` para visualizar información tabular.
- `MatDialog` para formularios, confirmaciones o información.
- `MatSnackBar` para mostrar mensajes al usuario.
- `MatProgressSpinner` o equivalente para representar estados de carga.

No es obligatorio utilizar todos los componentes anteriores. El equipo deberá seleccionar los que sean apropiados para su aplicación. La interfaz deberá funcionar correctamente tanto en computador como en dispositivos móviles.

3. Backend con FastAPI

El backend deberá desarrollarse utilizando FastAPI. Deberá implementar una **API REST** que permita al frontend consultar y modificar la información de la aplicación. Los datos enviados y recibidos deberán utilizar formato JSON.

4. Base de datos y ORM

La aplicación deberá utilizar PostgreSQL o MySQL.
El acceso a la base de datos deberá realizarse mediante un **ORM**, como SQLAlchemy, SQLModel u otro equivalente. El modelo deberá incluir como mínimo:

- usuarios;
- una entidad principal del sistema;
- al menos una relación entre entidades.

5. CRUD

La aplicación deberá implementar al menos un **CRUD completo** sobre una entidad principal:

- crear;
- listar y consultar;
- modificar;
- eliminar.

Todas estas operaciones deberán realizarse desde Angular consumiendo la API desarrollada con FastAPI.

6. Autenticación

La aplicación deberá contar con:

- registro de usuarios;
- inicio y cierre de sesión;
- contraseñas almacenadas de forma segura;
- autenticación mediante JWT;
- rutas protegidas en Angular mediante Guards;
- endpoints protegidos en FastAPI;
- envío automático del JWT mediante un Interceptor.

7. Estado y recursos reactivos

Se deberá utilizar **Signals de Angular** para manejar algún estado relevante de la aplicación. Por ejemplo:

- usuario autenticado;
- filtros o parámetros de búsqueda;
- selección actual;
- estado de la interfaz;
- información compartida entre componentes.

Además, se deberá utilizar al menos una de las APIs modernas de Angular para trabajar con recursos y datos asíncronos:

- `resource()`;
- `rxResource()`;
- `httpResource()`.

Esta herramienta deberá utilizarse en una funcionalidad real de la aplicación, por ejemplo para cargar un listado desde la API, obtener el detalle de un recurso, realizar una búsqueda dependiente de parámetros o actualizar información cuando cambie un Signal. Se espera que el equipo aproveche el comportamiento reactivo de Angular, evitando realizar manualmente operaciones que puedan resolverse mediante Signals y recursos reactivos.

8. Formularios

Los formularios principales deberán implementarse utilizando **Reactive Forms**. Se deberán utilizar validaciones apropiadas según los datos solicitados, mostrando mensajes cuando existan campos inválidos. Como mínimo se deberán utilizar Reactive Forms en:

- registro o inicio de sesión;
- creación o edición de la entidad principal.

9. Estados de la interfaz

La aplicación deberá informar al usuario sobre el resultado de sus acciones.
Se deberán considerar estados como:

- cargando;
- operación realizada correctamente;
- error;
- listado sin resultados;
- formulario inválido.

Se recomienda utilizar componentes de Angular Material como:
`MatSnackBar`, `MatDialog` y `MatProgressSpinner`.

Organización del proyecto

El frontend y backend deberán mantenerse separados.
Una estructura posible es:

```
proyecto/
|
|-- frontend/
|   ‘-- src/
|       ‘-- app/
|           |-- components/
|           |-- pages/
|           |-- services/
|           |-- guards/
|           ‘-- interceptors/
|
|-- backend/
|   |-- main.py
|   |-- models/
|   |-- schemas/
|   |-- routers/
|   ‘-- database/
|
‘-- README.md
```

Esta estructura es solo una referencia. El equipo podrá utilizar otra organización mientras mantenga una separación clara de responsabilidades.

Restricciones

- No utilizar Firebase, Supabase u otro servicio que reemplace el backend solicitado.
- No almacenar contraseñas como texto plano.
- No implementar la autenticación únicamente en Angular.
- No utilizar archivos JSON o datos estáticos para reemplazar la persistencia en base de datos.
- Los componentes Angular no deberán realizar directamente toda la lógica de acceso a la API. Para ello deberán utilizarse servicios.

Entrega

Cada equipo deberá entregar el enlace al repositorio Git con la solución completa.
El repositorio deberá incluir un archivo `README.md` que indique cómo ejecutar:

- frontend;
- backend;
- base de datos.

No deberán incluirse contraseñas, secretos JWT, claves privadas ni otras credenciales dentro del repositorio.

Consideraciones

El equipo deberá ser capaz de explicar el funcionamiento de su aplicación, incluyendo el consumo de la API, el uso del ORM, la autenticación mediante JWT y las herramientas propias de Angular utilizadas en el frontend.

Se puede reemplazar/combinar el uso de Angular Material por Angular Aria.

Evaluación

La evaluación estará compuesta por dos partes:

- **Producto desarrollado: 60 % de la calificación final.**
- **Interrogación: 40 % de la calificación final.**

Producto desarrollado

Los siguientes indicadores corresponden a la evaluación de la aplicación entregada. Las ponderaciones indicadas corresponden a la calificación del producto, la cual representa un **60 % de la calificación final**.

RA2

Desarrollar sitios web con contenidos estáticos y dinámicos que cumplan con los estándares de la industria utilizando herramientas profesionales de amplio uso en el mercado.

- Desarrollar la aplicación utilizando Angular, componentes reutilizables y Angular Router. **(8 %)**
- Diseñar una interfaz responsive utilizando Angular Material y/o Angular Aria. **(7 %)**
- Implementar formularios mediante Reactive Forms, incorporando validaciones y mensajes al usuario. **(7 %)**
- Gestionar estados de la aplicación utilizando Signals de Angular. **(6 %)**
- Utilizar `resource()`, `rxResource()` o `httpResource()` para gestionar datos asíncronos. **(7 %)**

Ponderación RA2 dentro del producto: 35 %

RA3

Construir servicios de backend basados en una API web moderna que permita su consumo desde un cliente web o móvil.

- Construir una API REST funcional utilizando FastAPI. **(6 %)**
- Implementar un CRUD completo para la entidad principal de la aplicación. **(7 %)**
- Modelar y persistir los datos mediante un ORM y PostgreSQL o MySQL. **(7 %)**
- Implementar el registro e inicio de sesión almacenando las contraseñas de forma segura. **(6 %)**
- Implementar autenticación mediante JWT y proteger los endpoints correspondientes. **(6 %)**
- Validar los datos y manejar adecuadamente los errores de la API. **(3 %)**

Ponderación RA3 dentro del producto: 35 %

RA6

Aplicar herramientas TIC para el almacenamiento, la comunicación, la transmisión e intercambio de información de manera efectiva.

- Integrar Angular con FastAPI mediante solicitudes HTTP e intercambio de datos en formato JSON. **(8 %)**
- Organizar el acceso a la API mediante servicios de Angular. **(6 %)**
- Implementar Guards e Interceptors para gestionar rutas protegidas y autenticación. **(8 %)**
- Integrar correctamente el flujo entre interfaz, API y base de datos. **(8 %)**

Ponderación RA6 dentro del producto: 30 %

Interrogación

El docente seleccionará **a cualquier integrante del equipo** para realizar la interrogación. La calificación obtenida será aplicada al equipo completo.

La interrogación se realizará directamente sobre el proyecto entregado. El docente podrá seleccionar cualquier proceso, funcionalidad o fragmento de código para solicitar su explicación.

- **Explicar** el funcionamiento de un proceso o fragmento de código seleccionado por el docente, identificando los elementos involucrados y su propósito. **(40 %)**
- **Relacionar** los distintos componentes involucrados en un flujo de la aplicación, explicando el intercambio de información entre Angular, FastAPI y la base de datos. **(35 %)**
- **Justificar** las decisiones de implementación y el uso de las tecnologías empleadas en el proceso seleccionado. **(25 %)**

Niveles de la escala de apreciación

Cada indicador del producto será evaluado utilizando los siguientes niveles de desempeño:

- **Excelente (1.0):** cumple completamente con el indicador, presentando una implementación correcta y funcional.
- **Bueno (0.6):** cumple con el indicador, presentando errores menores que no impiden su funcionamiento.
- **Regular (0.3):** cumple parcialmente con el indicador o presenta errores que afectan su funcionamiento.
- **Insatisfactorio (0.0):** no cumple con el indicador o la funcionalidad solicitada no se encuentra implementada.